



Laboratório de Banco de Dados no Colab

Aprenda MongoDB linha por linha criando o código do zero!

 **Regra do Laboratório:** Crie uma NOVA CÉLULA DE CÓDIGO no Google Colab para cada passo abaixo. Digite o código, leia a explicação, aperte o "Play" e veja a mágica acontecer!



PASSO ZERO: Preparando o Terreno

O Google Colab não vem com o nosso simulador de banco de dados instalado. Vamos baixar as ferramentas que precisamos.

```
# =====
# PASSO ZERO: Instalação da biblioteca que simula o MongoDB (Obrigatório no Colab)
# =====
!pip install mongomock
!pip install pymongo

import mongomock
from pymongo.errors import PyMongoError

print("=====")
# 🚀 INICIANDO O LABORATÓRIO PRÁTICO DE BACK-END
print("=====
")
```

O que fizemos? O comando `!pip install` diz ao servidor do Google para baixar e instalar o nosso "simulador de MongoDB" (mongomock). Em seguida, importamos essa ferramenta para dentro do nosso código.



BLOCO 1: Conexão (Abrindo o Armazém)

```
# -----
# 🖱️ BLOCO 1: CONEXÃO (Abrindo o armazém na memória do Colab)
# -----
# Em vez de ligar para um servidor externo, criamos o banco na memória agora
cliente = mongomock.MongoClient()
db = cliente['loja_banco']      # Criando a pasta do Banco de Dados
produtos = db['produtos']      # Criando a gaveta (Coleção) de produtos

print("🖱️ [CONEXÃO]: Simulador do MongoDB iniciado com sucesso!")
```

Conceito: O `MongoClient()` cria nosso banco de dados. Separamos o sistema de forma lógica: o banco `loja_banco` e dentro dele a coleção `produtos` (que é como se fosse uma gaveta onde só entram as fichas dos produtos).

✨ BLOCO 2: CREATE (Salvar a primeira ficha)

```
# -----  
# ✨ BLOCO 2: CREATE (Salvar a primeira ficha no banco)  
# -----  
novo_item = {  
    "nome": "Teclado Mecânico Gamer",  
    "preco": 189.90,  
    "estoque": 25  
}  
  
# Enviando para a gaveta de produtos  
retorno = produtos.insert_one(novo_item)  
  
print(f"✅ [CREATE]: Item salvo! O banco carimbou o ID único: {retorno.inserted_id}")
```

Conceito: O NoSQL não usa tabelas fixas. Ele salva dados no formato de dicionários (JSON). O comando `insert_one()` salva esses dados e o banco cria automaticamente uma identidade única chamada `_id`.

👁️ BLOCO 3: READ (Consultar a Gaveta)

```
# -----  
# 👁️ BLOCO 3: READ (Olhar e listar o que foi guardado)  
# -----  
print("\n--- 📋 [READ]: CONSULTANDO OS ITENS DA GAVETA ---")  
  
# O find() traz todas as fichas que encontrar  
todos_produtos = produtos.find()  
  
for p in todos_produtos:  
    print(f"👉 Produto: {p['nome']} | Preço: R$ {p['preco']} | Estoque:  
{p['estoque']}")
```

Conceito: O comando `find()` procura e devolve todas as fichas guardadas na coleção. Como pode haver milhares, usamos o `for` para ler uma por uma e mostrar no terminal de forma organizada.

🔄 BLOCO 4: UPDATE (Atualizar Dados)

```
# -----  
# 🔄 BLOCO 4: UPDATE (Mudar o valor de um dado existente)  
# -----  
filtro_busca = {"nome": "Teclado Mecânico Gamer"}  
nova_informacao = {"$set": {"preco": 219.90}} # O $set muda APENAS o preço  
  
produtos.update_one(filtro_busca, nova_informacao)  
print("\n🔄 [UPDATE]: O preço do Teclado foi reajustado no banco!")  
  
# Conferindo se mudou mesmo:  
produto_conferido = produtos.find_one(filtro_busca)  
print(f"🔍 [CONFERÊNCIA]: Novo preço verificado no banco: R$  
{produto_conferido['preco']}")
```

Conceito: Para atualizar, não podemos simplesmente jogar um dado novo por cima. Precisamos de duas coisas: achar o alvo (`filtro_busca`) e dizer o que mudar usando o `$set`. Isso garante que o estoque e o nome não sejam apagados por acidente!

⚡ BLOCO 5: OTIMIZAÇÃO (O Modo Turbo)

```
# -----  
# ⚡ BLOCO 5: OTIMIZAÇÃO (Ativando o "Modo Turbo" com Índices)  
# -----  
# Cria um sumário/índice para a coluna de preços, acelerando buscas futuras  
produtos.create_index([("preco", 1)])
```

```
print("\n⚡ [OTIMIZAÇÃO]: Índice de preços criado! Consultas agora são instantâneas.")
```

Conceito: Bancos de dados crescem. Se não avisarmos ao banco para criar um "sumário" (o comando `create_index`), ele vai procurar o preço linha por linha. O índice organiza e otimiza a busca para milissegundos.

🔒 BLOCO 6: TRANSAÇÃO (O Botão de Segurança)

Este bloco tem uma particularidade: bancos simulados em memória não suportam o recurso avançado de Sessões. Veja como o código seria:

```
# -----
# 🔒 BLOCO 6: TRANSAÇÃO (O Botão de Segurança "Tudo ou Nada")
# -----
print("\n🔒 [TRANSAÇÃO]: Iniciando uma simulação de venda segura...")

# Adicionando um mouse rapidamente para testar a venda
produtos.insert_one({"nome": "Mouse Sem Fio", "preco": 50.0, "estoque": 5})

# Abrindo uma sessão trancada com o banco
# A biblioteca mongomock não suporta sessões, então este bloco será comentado
# with cliente.start_session() as sessao:
#     with sessao.start_transaction():

#         # Passo 1: Diminuir 1 unidade do estoque do Mouse
#         produtos.update_one({"nome": "Mouse Sem Fio"}, {"$inc": {"estoque": -1}},
# session=sessao)

#         # Passo 2: Criar a nota fiscal/recibo na coleção 'vendas'
#         db['vendas'].insert_one({"item": "Mouse Sem Fio", "qtd": 1, "status":
"Pago"}, session=sessao)

#         print("🎉 [TRANSAÇÃO]: Sucesso! Estoque reduzido e recibo gerado juntos.")

print("⚠️ [AVISO]: O BLOCO DE TRANSAÇÃO foi pulado, pois mongomock não suporta sessões.\nPara usar transações, é necessário um servidor MongoDB real.")

print("\n=====")
print("🏁 FIM DO LABORATÓRIO: TODOS OS BLOCOS FUNCIONANDO!")
print("=====")
```

Conceito de Transação (ACID): Na vida real, a transação funciona como um pacote: ou você tira o item do estoque E gera o recibo ao mesmo tempo, ou o sistema cancela tudo em caso de falha, impedindo que o cliente pague por algo sem estoque.